Control Systems Technologies
Project Theme n°1: Elevator Control

Giorgio Ripani	Andrea Gallo	Francesco Tassi	Leonardo Lodolini
0001124508	0001117356	0001131551	0001116950

January 2026

1 Introduction and Policy

This report explains the functioning of the control system of an elevator programmed in codesys. It has been done using the GA approach, it includes the policy and three different GAs: Doors_GA, Leds_GA and Motor_GA.

At the beginning the program starts with the initialization (from "init" block to "doors_open" block). The blocks related to the Door_GA and to the Motor_GA (such as "doors_init" and "destination") are build in the same way, because they're both Do/Done GAs. They follow the handshake policy structure. The LEDs_GA are structured differently because they've the Start/Stop structure (LEDs_GA is never switched off, waiting for the operator). After the initialization if the program executed without faults it jumps to "start_led" block and execute again. If a fault happened the program "gets stuck" in the fault block waiting for an operator.

2 Motor GA

This GA is the biggest one of the project; it manages the movement of the elevator, the behavior in case of fault, and the initialization. It is equally divided in SFC and ST inside the blocks, but for conciseness of this report, only the main functions will be included.

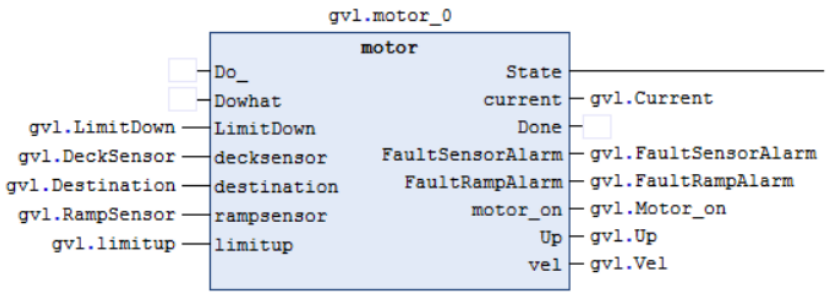

Figure 1: Motor Function Block main view.

The GA starts by putting itself in a `Ready` state. Then it is divided into three alternative choices: one for the initialization, one to go to destination, and one for when we get to the destination. Starting with the latter and simplest, it is only a block that sets `Done` and turns off the motor. Due to its simplicity, it won't be further explained.

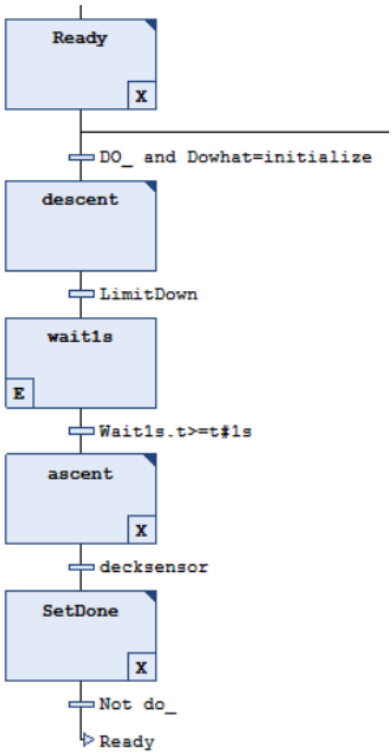

Figure 2: Initialization Sequence.

In each block, the necessary conditions of the motor, direction, and velocity are set as specified in the project directory.

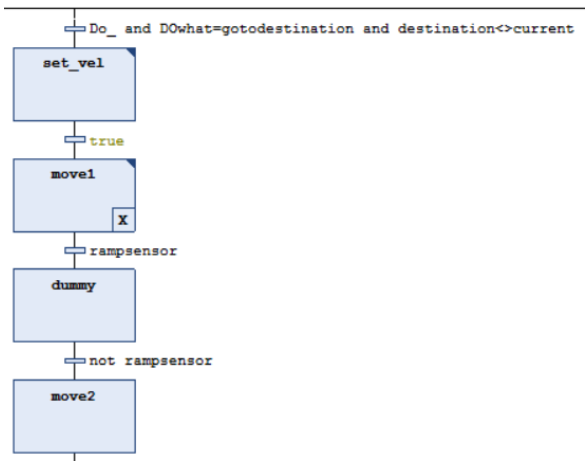

Figure 3: Logic for movement to destination.

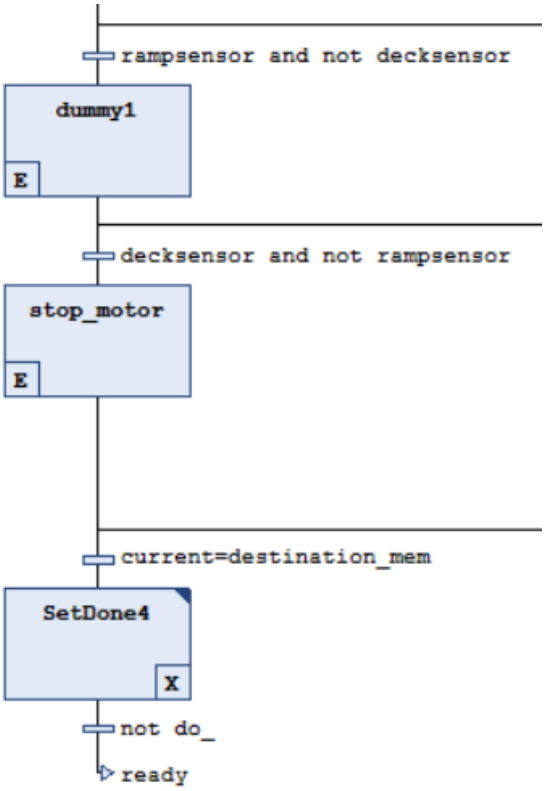

Figure 4: Detail of speed control.

Above is the scheme for the instruction "go to destination". The velocity is set to slow, then the block `move1` calculates the necessary direction based on the current position and the desired destination. It checks for the `rampsensor` and `NOT rampsensor` in order to avoid counting a ramp twice. Here we set an alternative path based on the next sensor the elevator meets (fault detection).

Let's assume we don't have a fault for now. After the second ramp sensor, the entry action in `dummy1` changes the current floor and slows down the elevator if we are approaching the destination. Then we stop the motor once we have reached the `decksensor`. The last block `SetDone` interacts with the policy to implement the handshake.

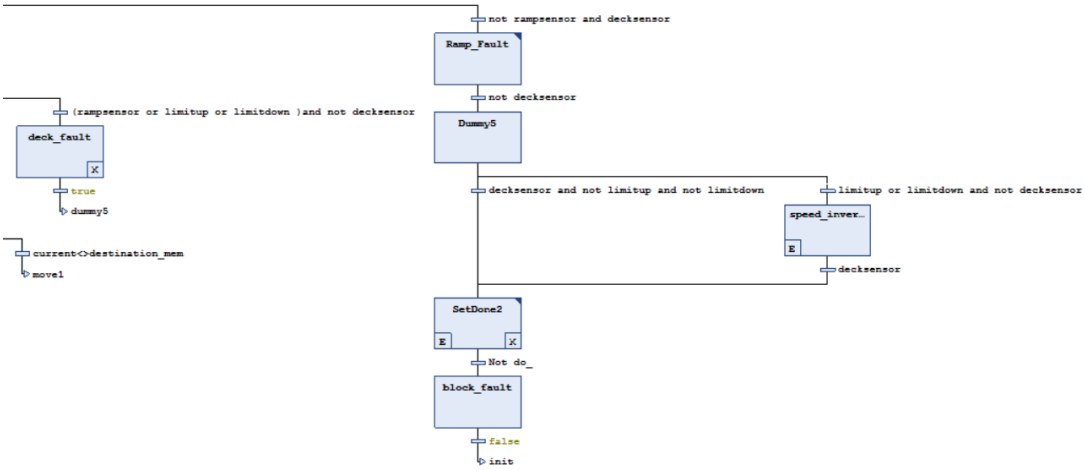

Figure 5: Alternative paths for sensor logic.

2.1 Fault Detection & Handling

Now let's consider the alternative choices shown above. Starting from the left: `DeckFault` is found if, instead of the `decksensor`, the elevator meets a `rampsensor` or a limit (`Up` or `Down`) in the case of the floors at the extremes. The fault is then handled in the other path.

The other path detects `rampfaults`, having detected a `decksensor` where a ramp should have been. So both `deckFault` and `RampFault` lead to `dummy5`. Here we have two alternative options: if we run into a limit sensor, we invert the direction and keep the motor running until we reach a `decksensor`; in the easier case in which we first run into a `decksensor`, the motor stops completely until the fault gets fixed.

3 Door GA

This GA is aimed at controlling the elevator's door behavior. The structure is both simple and straightforward: after the initial step of initialization and the second that sets the state as `Ready` (and, as exit action, `Busy`), an alternative choice opens.

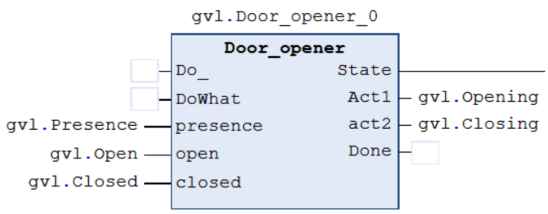

Figure 6: Door Function Block.

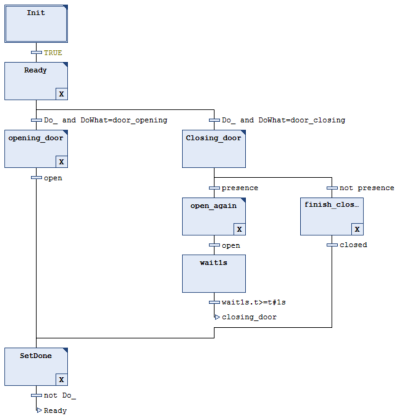

Figure 7: Door SFC Scheme.

The **left side** controls the opening action of the door: if the policy sends the opening signal, the `opening_door` block is executed; it sets `Act1` (actuator that opens the door when activated) as `TRUE`, so the door opens. When this action is accomplished, the `Open` sensor is activated and, as exit action, `Act1` is set as `FALSE` so the motor is switched off.

The **right side** controls the closing action of the doors: if the policy sends the closing signal, the `closing_door` block is executed. This block checks if the `Presence` sensor isn't activated; if it is the case, it sets `Act2` as `TRUE`, activating the motor that closes the door. After that block, another alternative choice is found. If the presence sensor is still not triggered, the `finish_closing` block is executed and the doors are successfully closed; as exit action, it sets the `Act2` motor as `FALSE`, switching off the closing door motor.

If, instead, while the doors are closing, the `Presence` sensor is triggered, the `open_again` block is activated. This block sets `Act1` as `TRUE` again and, consequently, the door opens. When this happens, the `Open` sensor is activated; as exit action, `Act1` is set `FALSE`, switching off the opening door motor. Then the `waits` block is executed. After a 1-second timer, there's a jump at the `closing_door` block, starting the closing cycle again until the `Presence` sensor isn't triggered.

After the door is successfully opened or closed, the `SetDone` block is activated; this changes the variable `Done` as `TRUE` for the policy. When the policy sends the signal not `Do`, the exit action is activated and `Done` is set `FALSE`, successfully completing the handshake protocol, and the control jumps to the `Ready` block waiting to start again the sequence explained above.

4 Leds GA

The HMI has a dedicated GA that manages the LED lights inside the elevator and at the various floors. Also, and not less important, it takes care of the sequence of calls and stops to be done. Once the GA is called from the policy, it keeps working throughout the whole program execution unless otherwise stated in the policy.

Below is the Function Block of the said GA. The GA is centered on a ST algorithm, included in a very simple SFC scheme.

Figure 8: HMI Function Block.

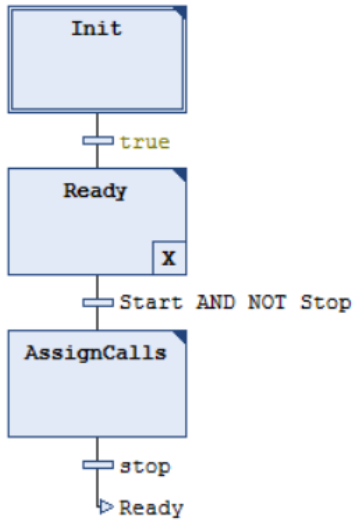

Figure 9: HMI SFC Scheme.

The algorithm is inside the `AssignCalls` Block.
Let's briefly explore its functioning. Using an internal variable `ARRAY [0...7] OF BOOL`, we assigned every floor to a cell of the array, taking into consideration all the relative buttons (internal panel and external Deck). Using a `FOR` cycle, we scan the array checking for the active floors. For the booked floors (`arrButtons[z]=1`), we turn on the LED of the pressed button.

Then, since the array `arrButtons` is not persistent (as soon as the button is not pressed anymore it'd reset), we cloned the calls in a different array, where the call is latched until the elevator reaches the destination floor.

That array gets scanned in order to not write twice the same call, then writes in a separate variable the destination with the highest priority. After that, we check if the elevator reached its destination, in which case the light gets turned off. Lastly, we shift the elements of the array `destinations` to define the new priority of the calls.